

< Robotique TEK5 · Day 1 · Afternoon />

Kickoff - Project: LeRobot



< Afternoon agenda />

Morning: Theory

Hugging Face & LeRobot

The project: architecture & hardware

Live demo + RViz

Tomorrow: Docker build, then free demo

ROS2 :From plumbing to robot



< This morning → this afternoon />

This morning: the plumbing

- Nodes, topics, services, actions
- Rate (hz) & QoS
- TF & RViz2
- colcon, Docker
- Nav2, MoveIt 2 (outlook)



This afternoon: the robot

- A real arm: SO-101
- Its ecosystem: LeRobot
- Your project: perception → pick & place
- Live demo
- Your own application



ROS 2 to connect the building blocks — hardware & software



Hugging Face & LeRobot



< Hugging Face: the GitHub of AI />

Models

Over a million open-weight models:
LLMs, vision, audio and robot policies.

Datasets

Versioned, shared data. In robotics:
demonstration episodes.

The screenshot shows the LeRobot organization page on Hugging Face. The header includes the Hugging Face logo, a search bar, and navigation links for Models, Datasets, Spaces, Buckets, Docs, Enterprise, Pricing, Log In, and Sign Up. The LeRobot organization card is prominently displayed, featuring the LeRobot logo, the tagline 'Open-source AI for real-world robots', and a description: 'LeRobot is Hugging Face's library for teaching robots by example. Guide a robot through a task a few times and it learns to do it on its own, from 3D-printed arms to full humanoids.' Below the description are links for Documentation, GitHub, Discord, and @LeRobotHF. The page also features a 'Recent Activity' section with updates from users like CarolinePascal, mishig, and pepijn223. A 'Papers' section lists 'Robot Learning: A Tutorial'. The 'Team members' section shows 17 members. The 'How it works' section outlines a four-step process: 1. Teleoperate, 2. Record, 3. Train, and 4. Deploy, each with a brief description of the step.

Organization Card

LeRobot

Open-source AI for real-world robots

LeRobot is Hugging Face's library for teaching robots by example. Guide a robot through a task a few times and it learns to do it on its own, from 3D-printed arms to full humanoids.

[Documentation](#) [GitHub](#) [Discord](#) [@LeRobotHF](#)

Recent Activity

- CarolinePascal updated a bucket 2 days ago
- mishig updated a Space 3 days ago
- pepijn223 new activity 4 days ago

Papers

- Robot Learning: A Tutorial

Team members 17

How it works

Robots learn from your demonstrations, the same four steps on every robot.

- 1 Teleoperate**
Move the robot yourself with a leader arm, phone, or gamepad.
- 2 Record**
Each run is saved as a dataset: camera video plus every movement.
- 3 Train**
Train a model that controls the robot. Use your own GPU or the cloud.
- 4 Deploy**
Run it on the robot and watch it work on its own.

Why it matters

- Open-source
- Since 2024: a robotics branch, LeRobot
- Reproducible: code + data + weights

< LeRobot: open-source robotics />

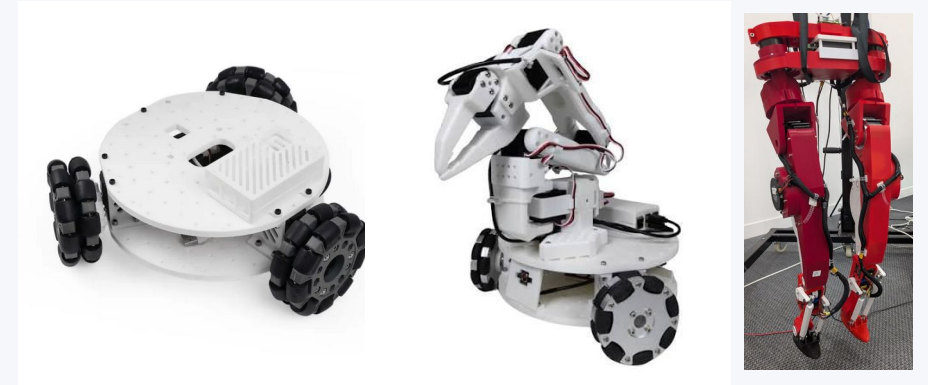
Data

LeRobotDataset format:
videos + states + actions



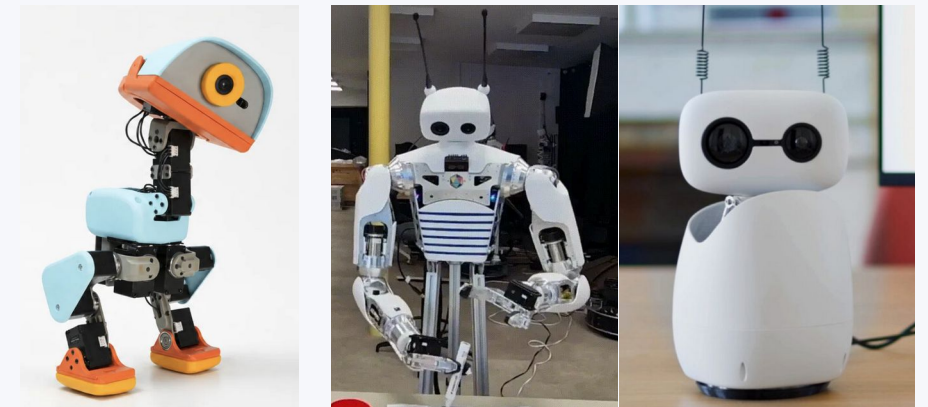
Policies

ACT, Diffusion Policy, SmoIVLA, π_0
trainable and deployable

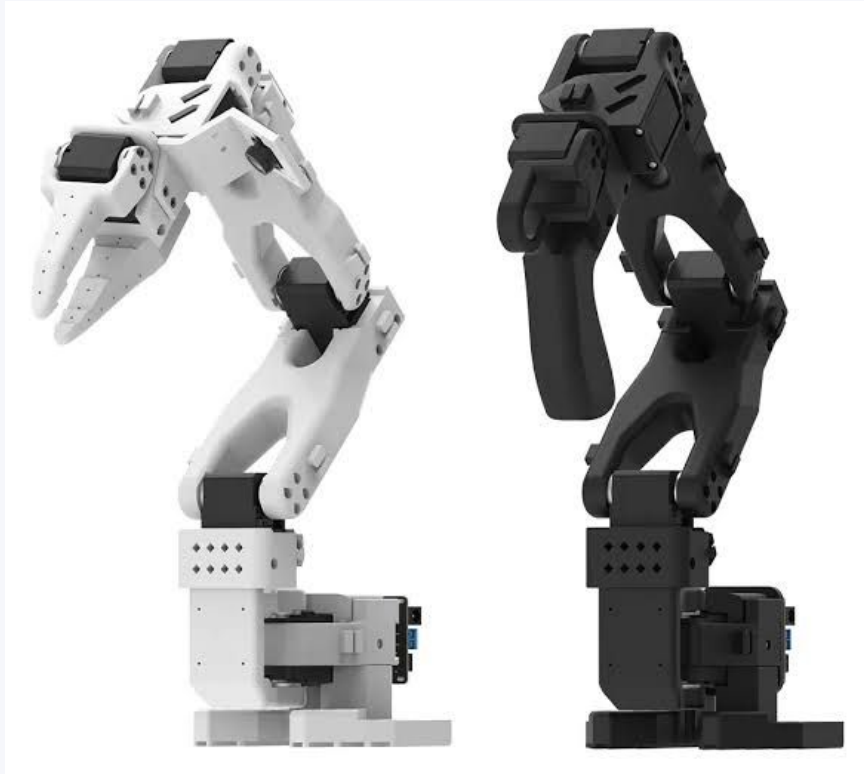


Robots

Ready-made drivers:
SO-100/101, LeKiwi, Koch, Reachy...



< The SO-101 arm />



6

Feetech STS3215 servos

5 + 1

DoF + gripper

3D

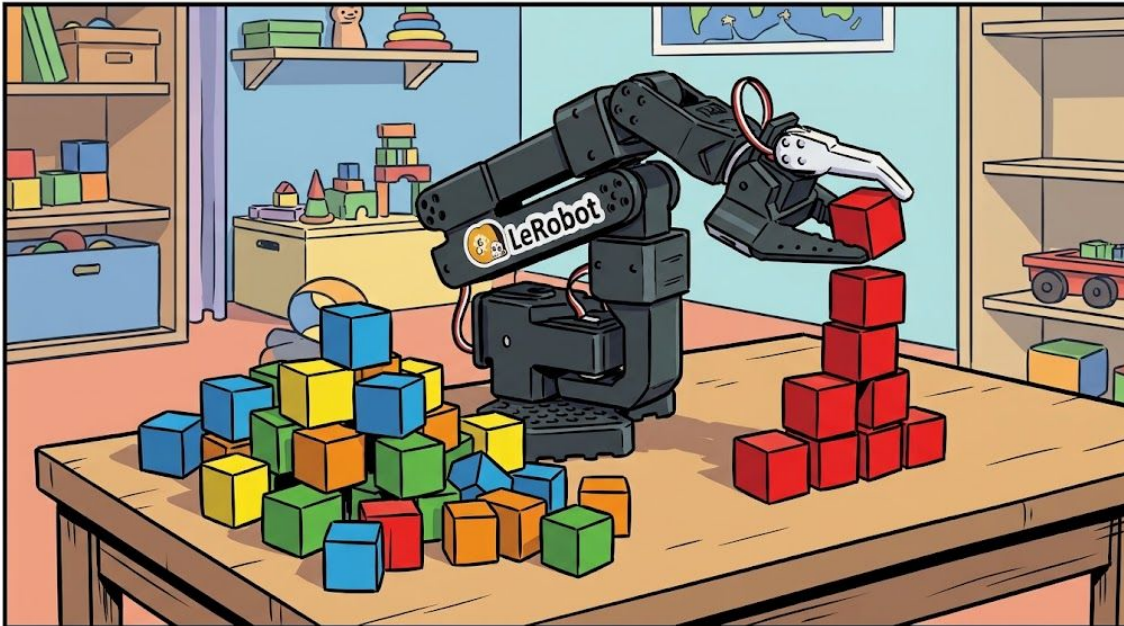
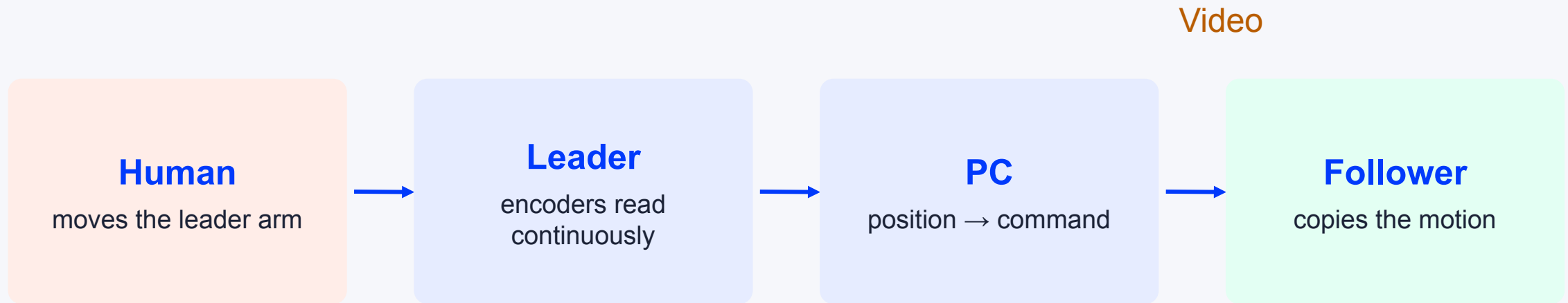
printable parts, open-source design

→ Don't hesitate if you want to modify the design !

~250 €

Per kit (2 arms)

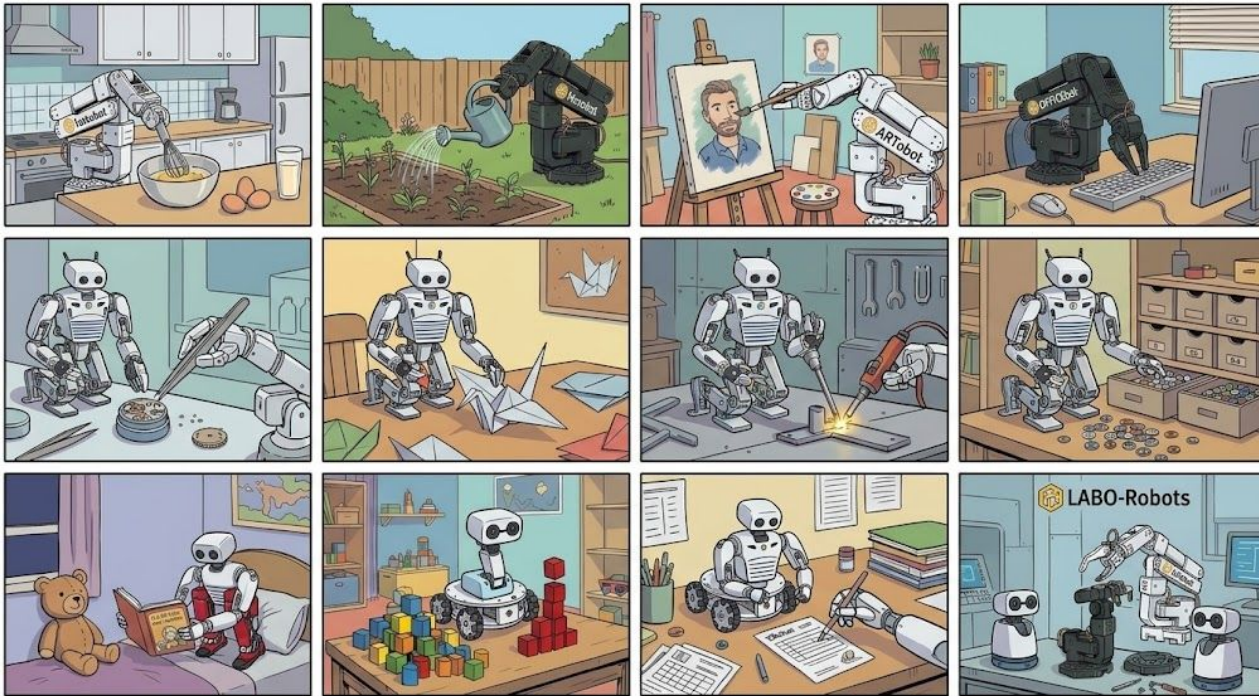
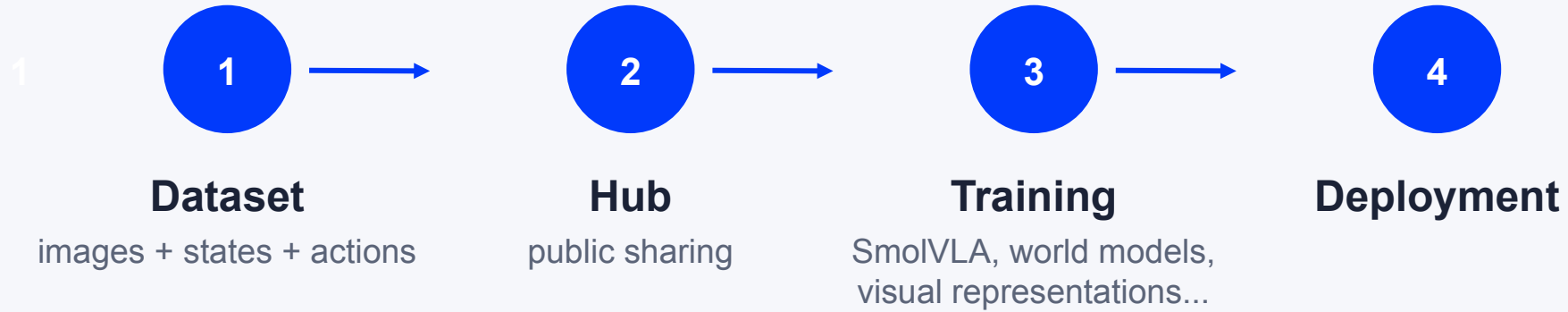
< Leader / Follower: teleoperation />



What is it for?

- Want the robot to do a task without coding ?
- Record demonstrations
- Build a dataset
- Robot autonomously does the task !

< From motion to model />



Key takeaways

- **Open-source:** Anyone can collect and share
- **Foundation-ready:** Community scale turns niche tasks into general-purpose models
- **Accessible robotics:** From an affordable arm at home to state-of-the-art policies



What about this project ?



< Same arm, two approaches />



Data-driven *with leader arm*

demonstrations → policy

- Do complex tasks
- Data collection & model training
- “Magic” AI (implicit model learned from data) ...
- ... but hard to debug

Model-based *(our project)*

perception → IK → control

- Control & Understanding
- Predictable behavior
- No data needed ...
- ... but complex to model (*or maybe not!!*)

In reality, both approaches are complementary, not competing

< This Robotics Project />

PHASE 1



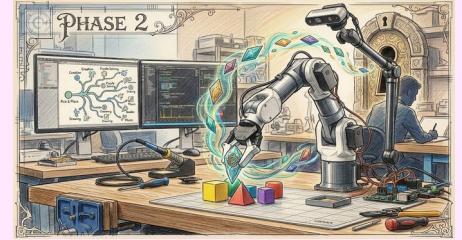
Robotics skills

- Pick & place model-based
- **ROS 2** with **perception**, **IK**, **control**
- Sim, then real arm

Mandatory

Note: AI for perception is ok if you think it's needed

PHASE 2



Exploration

- A new task with your arm, beyond pick & place
- Reuse your phase 1 pipeline
- Push one block further (perception, control, AI...)

Once phase 1 is validated

Understand the robot before handing it to an AI model

	Group A - October	Group B - Late November
Kickoff + Bootstrap	October 8-9	November 26-27
<i>Online class #1</i>	October 22	December 10
Mid-term review	~October 31	~December 19
<i>Online class #2</i>	November 5	January 7
Final review	November 21 or 28	January 23 or 30

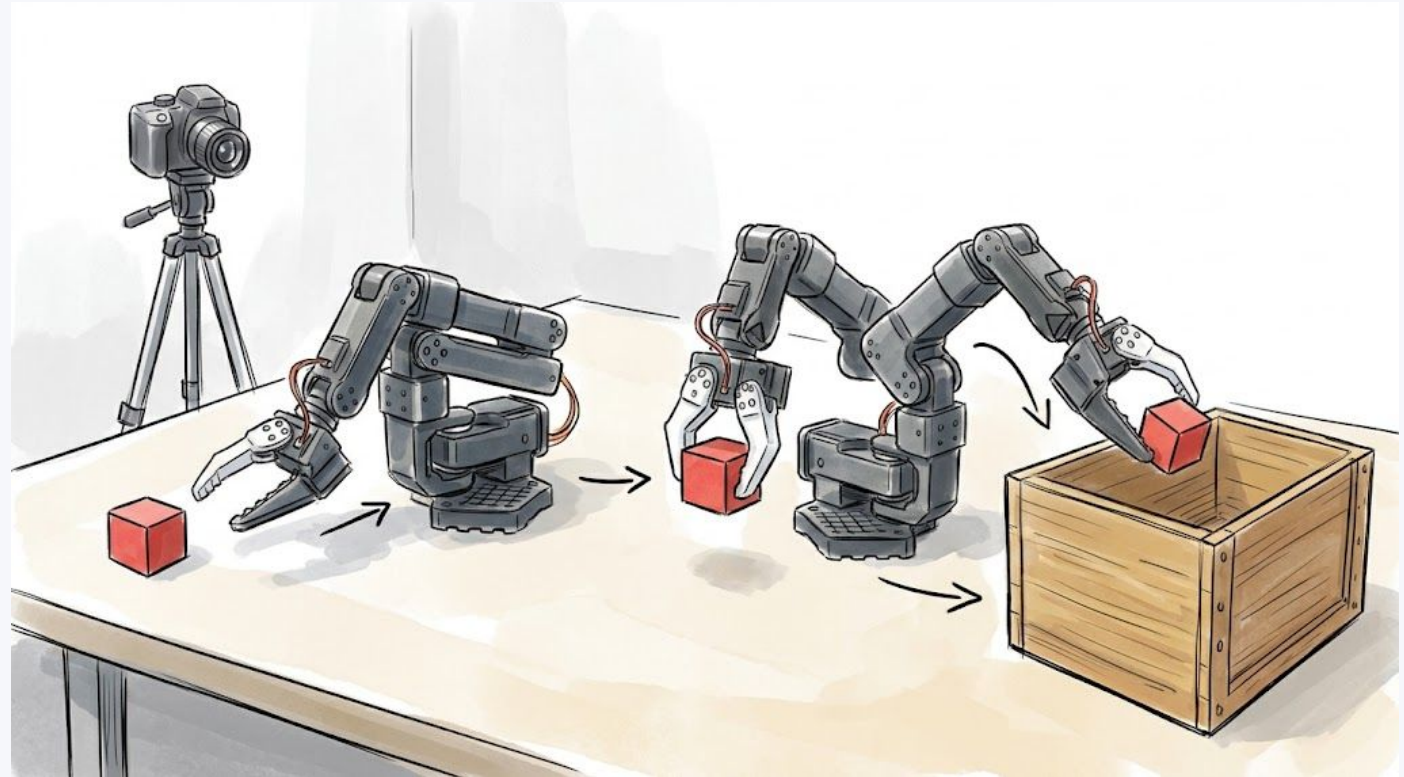
The project: pick & place



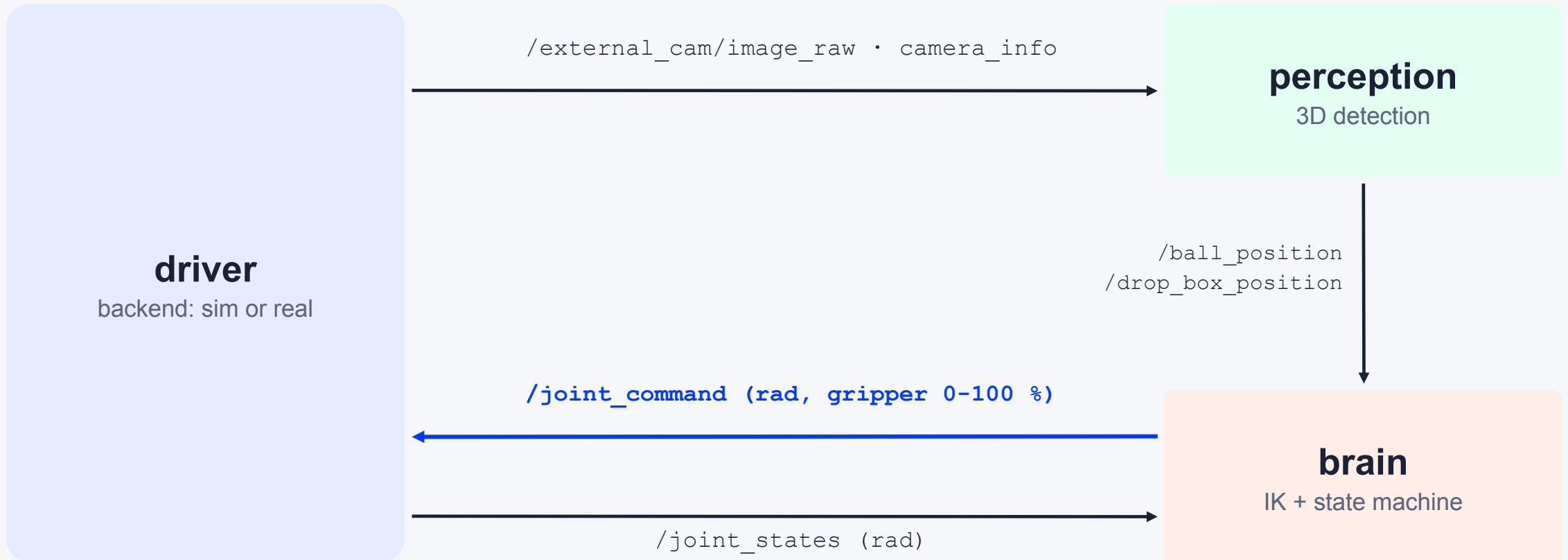
< The goal />

See the object.
Grasp it.
Drop it in the box.

... then go beyond !



< Overall architecture />



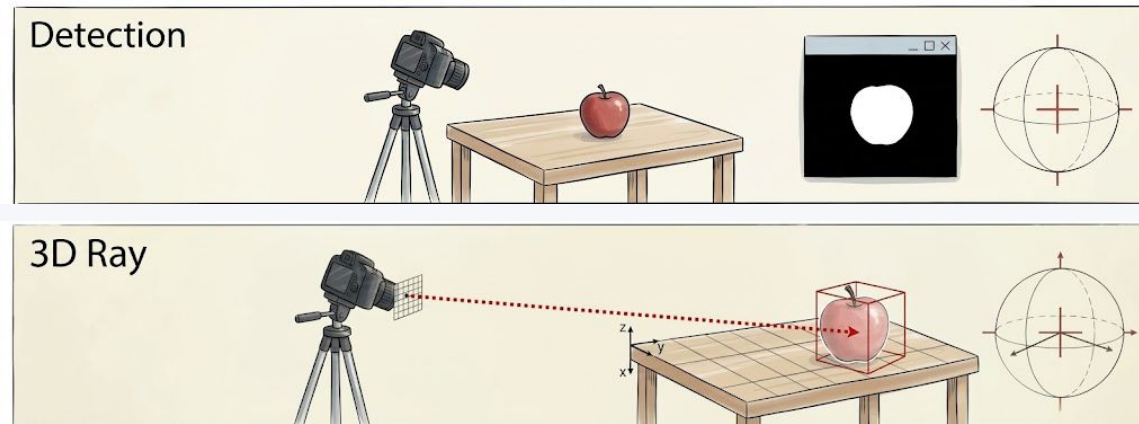
Backend: SO101Sim (MuJoCo) or SO101Follower (real arm) · `param use_sim`

< Topics you need to know : ros2 topic list />

Topic / service	Type	From → to	Notes
/external_cam/image_raw	sensor_msgs/Image	driver → perception	camera stream
/external_cam/camera_info	sensor_msgs/CameraInfo	driver → perception	intrinsics K
/ball_position	3D position	perception → brain	robot frame
/drop_box_position	3D position	perception → brain	drop target
/joint_states	sensor_msgs/JointState	driver → brain, RSP	rad (REP-103)
/joint_command	sensor_msgs/JointState	brain → driver	rad + gripper 0-100 %
/brain/start • /brain/stop	std_srvs/Trigger	you → brain	service

RSP = robot_state_publisher

< Node perception & visualization on Rviz2 />



Inputs

`image_raw`
`camera_info`

Outputs

`/ball_position`
`/drop_box_position`

Visualisation

mask image (segmented object)

1

Detect in 2D

color thresholding (HSV), contours, centroid

2

Pixel → 3D ray

intrinsics K from camera_info

3

Ray → point

table plane or known object size

4

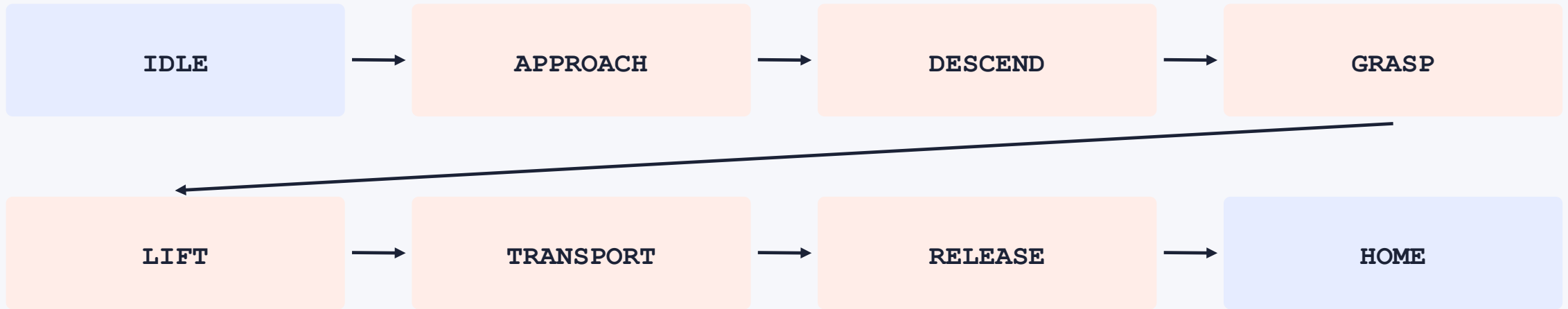
Camera → robot

extrinsics, via TF

Video Yolo

Where is your camera in the 3D world ?
This camera doesn't see depth... so how do you do ?

< Node brain />



Example state machine

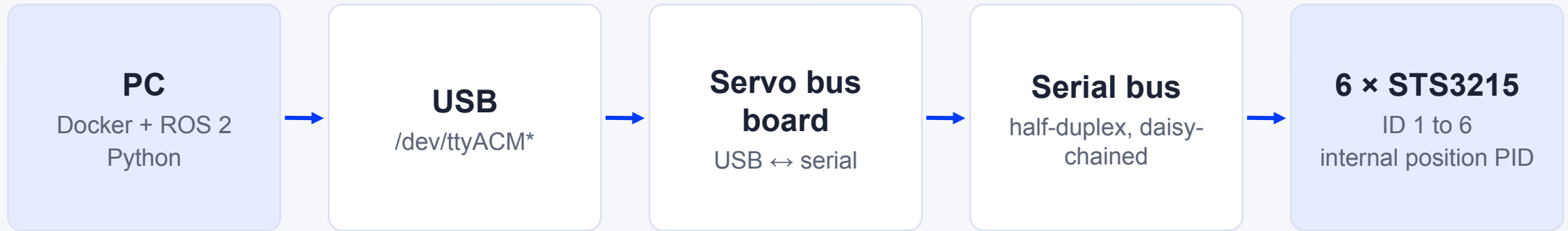
Inverse kinematics

- Target position → joint angles (ikpy)
- 5 DoF: no full orientation
- Several solutions, or none

State machine

- One state = one simple goal
- Transitions on measured conditions
- Publish the state on /brain_state

< From PC to motor />



Pitfalls (for tomorrow)

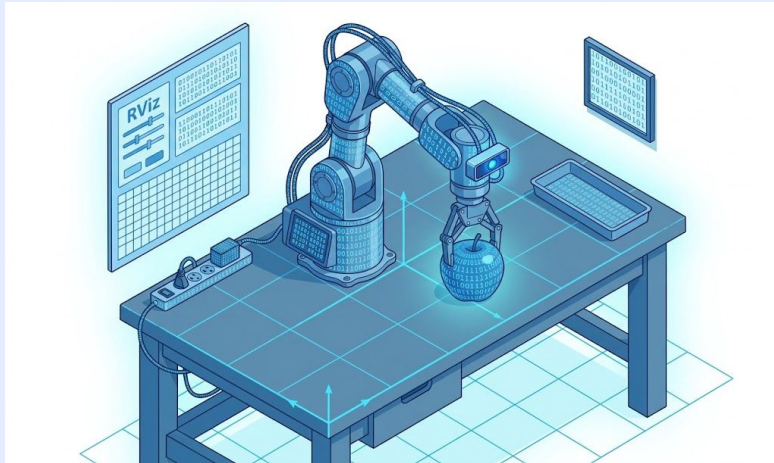
- Find the right port: ls /dev/ttyACM*
- Pass the device to the Docker container
- Port permissions (dialout group)
- Calibration: offsets and joint directions

< Sim ↔ real: one parameter />

SO101 - Simulation

MuJoCo

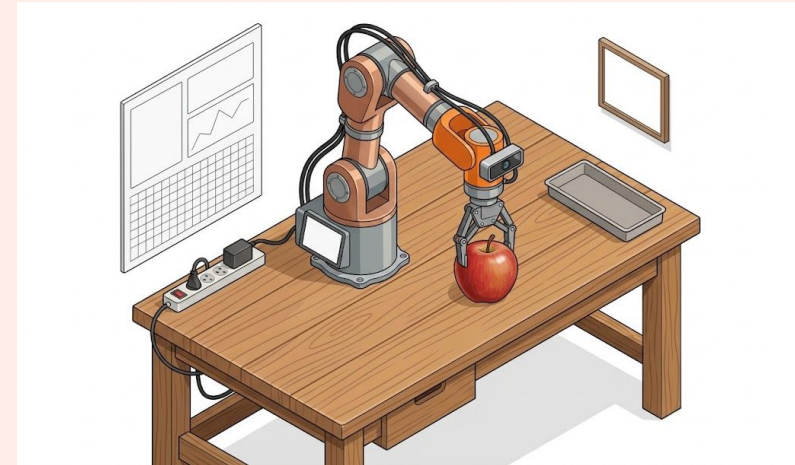
- Fast, risk-free iteration
- ~ Perfect camera and scene
- No calibration



SO101 - Real

The real arm

- Latency, noise, mechanical backlash
- Sim-to-real : Lighting, reflections, real camera ...
- Calibration required



Make it easy to switch from simulation to reality in your code

< Roadmap />

-
- 1 Environment**
Docker, sim, Rviz
- BOOTSTRAP -
 - 2 Move the arm**
publish on /joint_command
- BOOTSTRAP -
 - 3? Perception**
Calibration
object + box in robot frame
 - 4? IK**
reach a target position
 - 5 Pick & place**
In simulation
 - 6 Real arm**
 - 7 Exploration**
your application

Tomorrow morning: steps 1 and 2

{EPITECH}

<SINCE 1999/>

< Part 5 />

Live demo

{ <TECH TOGETHER TOMORROW> }



< What we'll watch />

1

The pipeline starts

docker compose up, wait for « Brain node prêt. »

2

We start the task

ros2 service call /brain/start

3

The arm executes

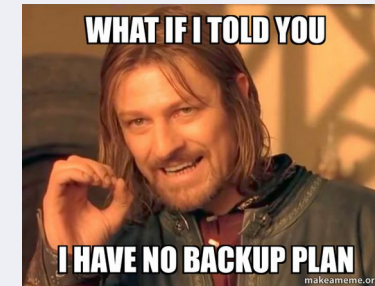
approach, grasp, transport, drop

4

RViz shows everything

model, TF, segmented image, brain state

 Backup video: full demo



< Reading RViz />

RobotModel `/robot_description + TF`

TF `robot_state_publisher ←
/joint_states`

Image `camera image / perception mask`

Fixed Frame `the robot base frame`

Arm in pieces? No `/joint_states`: driver not running.

 *Image: annotated RViz screenshot*





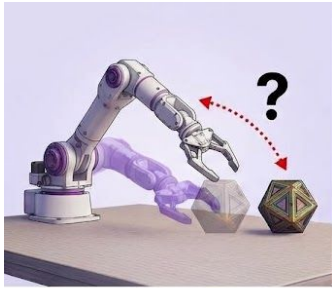
Is it robust?



< Ask yourselves these questions />

? The object moves mid-action

Recompute the new movement ?



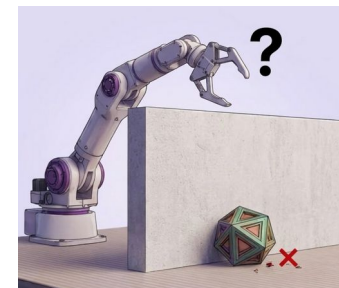
? The grasp fails

*How do you know?
What do you do?*



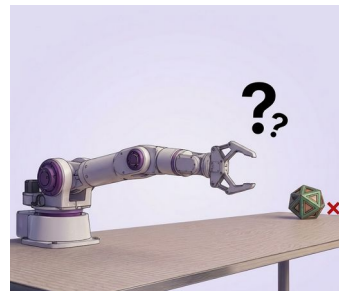
? The object leaves the view

*What does perception publish?
Is there a memory ?*



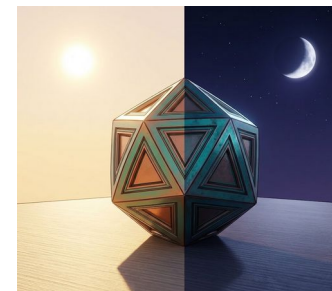
? Target out of reach

*What does IK return?
Does the arm move anyway?*



? Lighting changes

*Does your color threshold still hold?
Need a more robust perception ?*



{EPITECH}

<SINCE 1999/>

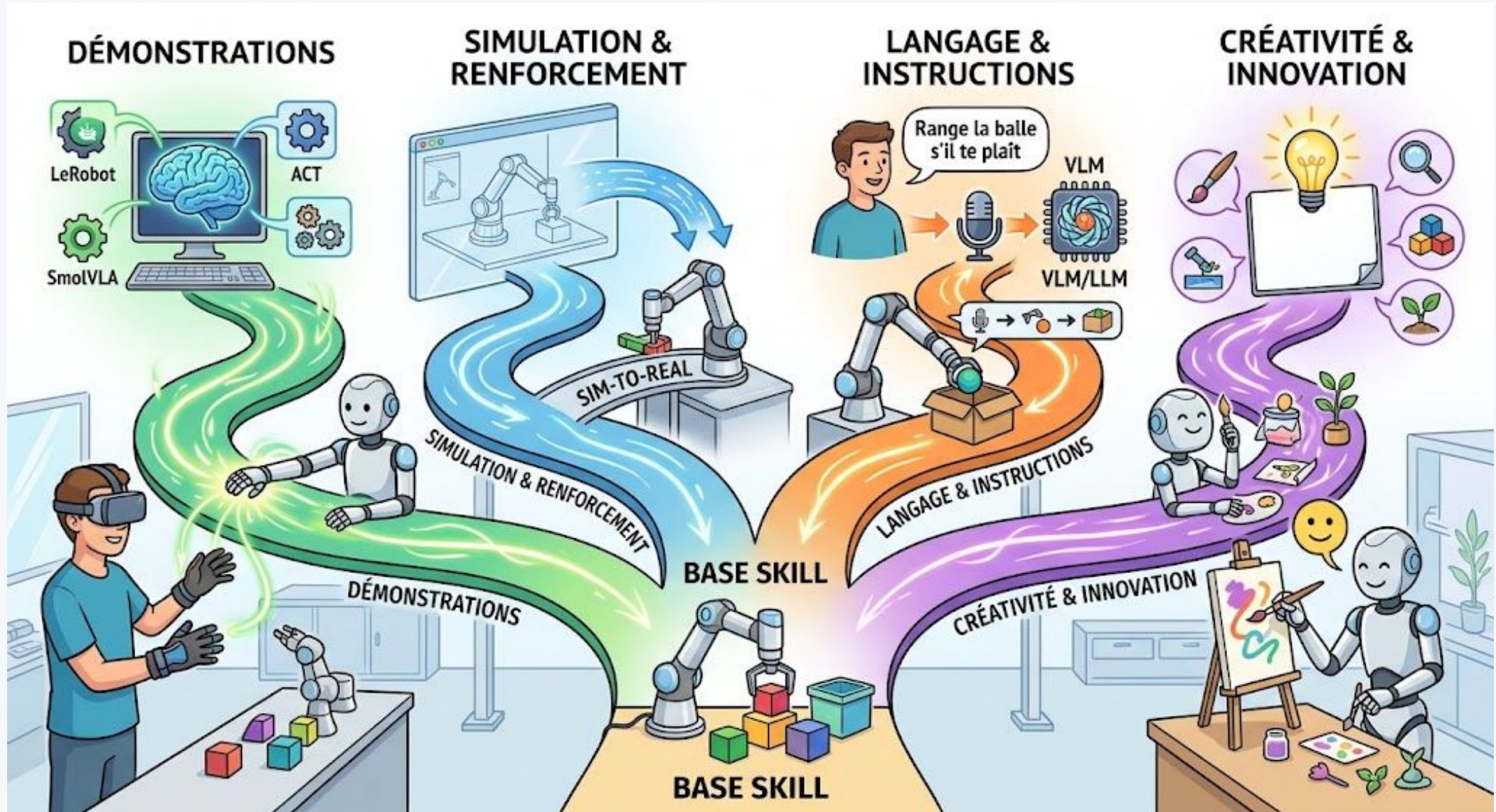
< Part 7 />

Going further

{ <TECH TOGETHER TOMORROW> }



< After pick & place />



< Why it matters: your resume />

What you will walk away with

- 30s demo video
- Clean, documented GitHub repo
- Valuable skills for your resume / CV
→ Required for robotics jobs
- Do something for you during the exploration :)

Skills demonstrated

ROS 2

3D perception

Inverse kinematics

Hardware

Experience Sim2real
issues

State machines

Docker



What about tomorrow?



< Tomorrow: the bootstrap />

MORNING

Guided hands-on

- Install: Docker, sim, RViz
- The real arm: checks, calibration
- Publish on /joint_command

→ **the arm moves**

AFTERNOON

Autonomous development

- Team work on the 3 nodes
- Technical support on demand
- Start with the sim

```
ros2 topic pub --once /joint_command sensor_msgs/msg/JointState '{name: [shoulder_pan], position: [0.5]}'
```

< Before you leave tonight />



Linux host

native or VM, with Docker



Clone the repo

tek5-robotics



Run the build

`cd docker && docker compose build`



Form groups (3 per group)

+ pick a name + Think about your project idea

Why tonight?

Don't nuke the school Wi-Fi tmw <3